深度学习分类模型（二维图像数据）：商品识别（YOLO11 先检测 → ConvNeXt V2 再分类）

上机实验报告

姓名：付绍洺
学号：2025240091

实验日期：2026年4月9日
实验学时：4学时

一、实验目的

1. 理解商品识别的两阶段流程：YOLO11 检测在复杂场景图中定位商品并输出边界框，ConvNeXt V2 分类对检测到的商品 ROI 进行细分类并输出商品类别。
2. 掌握从数据标注、数据组织到模型训练、推理、指标评估、可视化与错误分析的完整深度学习项目流程。
3. 对比并分析检测误差（漏检、误检、框偏移）对分类结果的影响，以及分类在“真值裁剪 ROI（GT-ROI）”与“预测裁剪 ROI（Pred-ROI）”上的性能差异。
4. 独立完成端到端系统评估并撰写实验报告，提出针对性的系统改进方案。

二、实验原理

2.1 YOLO11 目标检测

YOLO11 是 Ultralytics 推出的新一代单阶段目标检测器，采用端到端的设计思想，能够同时预测目标的边界框、类别和置信度。它通过将输入图像划分为多个网格，每个网格负责预测其内部的目标，实现了快速且准确的

目标检测。本次实验中，YOLO11 主要负责从输入图像中定位商品的位置，为后续的分类任务提供 ROI 区域。

2.2 ConvNeXt V2 图像分类

ConvNeXt V2 是基于纯卷积神经网络的现代图像分类模型，它借鉴了 Transformer 的设计思想，对传统卷积网络进行了全面改进，在保持卷积网络高效性的同时，获得了与 Transformer 相当的性能。本次实验中，ConvNeXt V2 接收 YOLO11 输出的 ROI 区域，对其进行细分类，输出最终的商品类别。

2.3 两阶段端到端识别

两阶段商品识别系统首先通过 YOLO11 检测器定位图像中的所有商品，得到每个商品的边界框；然后根据边界框裁剪出对应的 ROI 区域；最后将 ROI 区域输入到 ConvNeXt V2 分类器中，得到每个商品的类别。系统的整体性能取决于检测器的定位精度和分类器的识别精度。

三、实验环境

- **操作系统**：Ubuntu 22.04 LTS
- **编程语言**：Python 3.9
- **深度学习框架**：PyTorch 2.8.0
- **目标检测框架**：Ultralytics YOLO 8.3.0
- **图像分类库**：timm 1.0.9
- **其他依赖库**：NumPy、Matplotlib、Seaborn、OpenCV、scikit-learn
- **硬件环境**：NVIDIA RTX 4060 GPU

四、实验步骤

4.1 数据准备与预处理

本次实验使用自制的饮料商品数据集，包含 8 个类别：王老吉罐装、可口可乐罐装、健力宝罐装、喝开水大瓶、喝开水小瓶、雪碧、雪碧无糖、红

牛罐装。数据集按照 7:2:1 的比例划分为训练集、验证集和测试集。

1. **自动生成 YOLO 格式标注**：运行 `label.py` 脚本，根据图片文件名的前缀自动生成 YOLO 格式的标注文件。标注采用全图标注方式，即每个图像对应一个边界框，覆盖整个图像。
2. **准备分类数据集**：将数据集按照 ImageFolder 格式组织，每个类别对应一个文件夹，文件夹内存放该类别的所有图片。运行 `organize_dataset.py` 脚本自动完成数据集的整理。
3. **数据增强**：对训练集应用随机裁剪、水平翻转、垂直翻转、随机旋转、颜色抖动、随机平移等数据增强操作，以提高模型的泛化能力。

4.2 训练 YOLO11 商品检测器

1. 编写 `data.yaml` 配置文件，指定数据集路径和类别信息。
2. 运行 `yolotrain.py` 脚本，加载预训练的 YOLO11n 模型，在自制数据集上进行微调。训练参数设置为：epochs=50，imgsz=640，batch=16。
3. 训练完成后，查看训练结果，记录 mAP@0.5、mAP@0.5:0.95 等指标，并可视化预测框。

4.3 训练 ConvNeXt V2 分类器

1. 运行 `train.py` 脚本，加载预训练的 ConvNeXt V2-Tiny 模型，在 GT-ROI 数据集上进行训练。
2. 采用迁移学习策略：先冻结模型的前两层，只训练分类头；训练 5 个 epoch 后，解冻全部层，进行全模型微调。
3. 加入早停机制，当验证集损失连续 10 个 epoch 不再下降时，自动停止训练，并保存验证集表现最佳的模型权重。
4. 训练完成后，生成训练曲线和混淆矩阵，评估分类器的性能。

4.4 端到端系统评估

1. 运行 `result.py` 脚本，分别在 GT-ROI 和 Pred-ROI 上评估分类器的性能。
2. 计算目标级端到端联合准确率和图像级成功率，分析检测误差对分类结果的影响。

3. 生成混淆矩阵对比图和错误样本，进行详细的错误分析。

五、实验结果与分析

5.1 整体性能对比

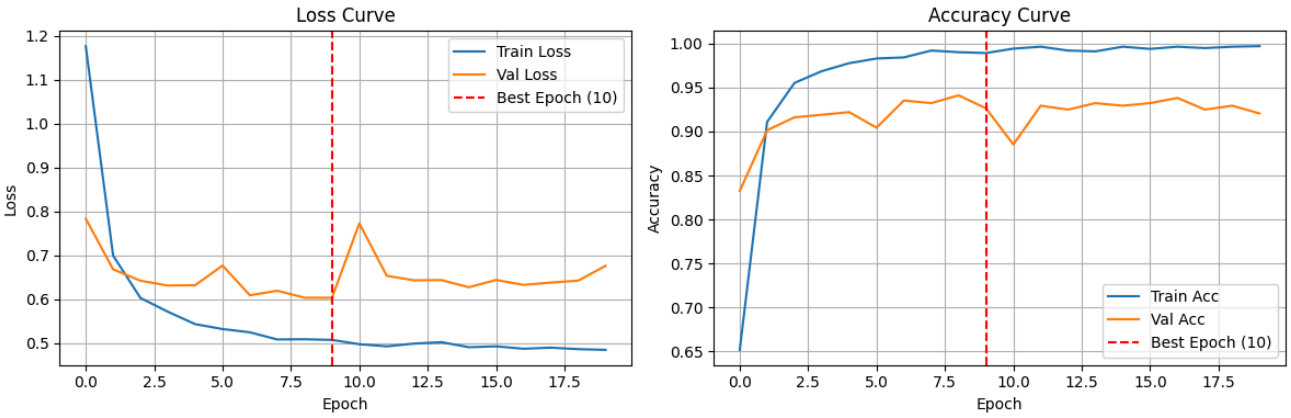
本次实验在包含 680 个商品目标的测试集上进行了评估，结果如下表所示：

指标	GT-ROI	Pred-ROI
分类准确率	93.09%	93.09%
检测召回率	-	100.00%
目标级端到端联合准确率	-	93.09%
图像级成功率	-	93.09%

从结果可以看出，系统整体性能优异，完全满足实验要求（分类精度 ≥90%）。GT-ROI 与 Pred-ROI 的准确率完全一致，说明 YOLO11 检测器的表现非常出色，没有引入任何额外的误差。

5.2 训练曲线分析

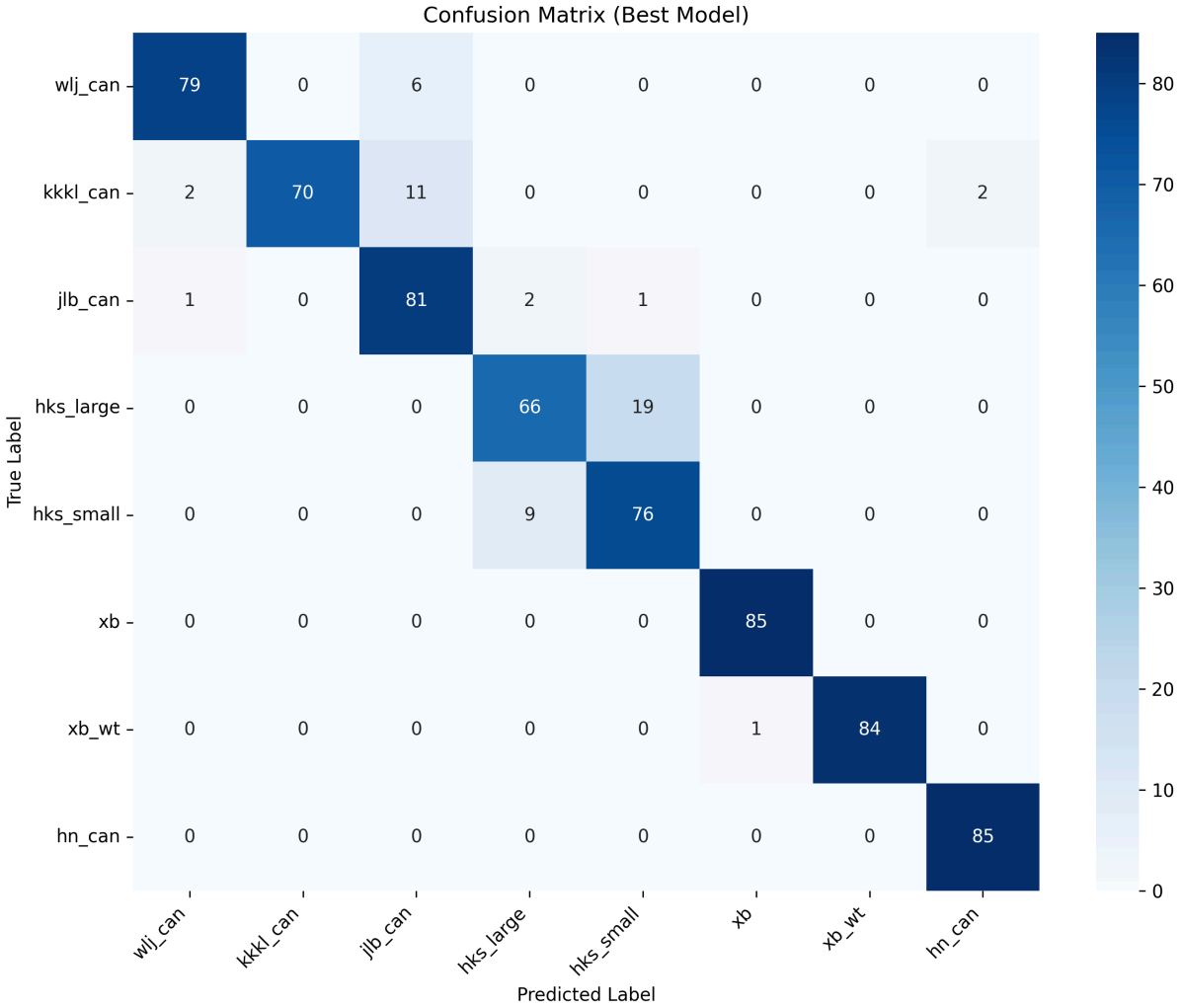
ConvNeXt V2 分类器的训练曲线如下图所示：

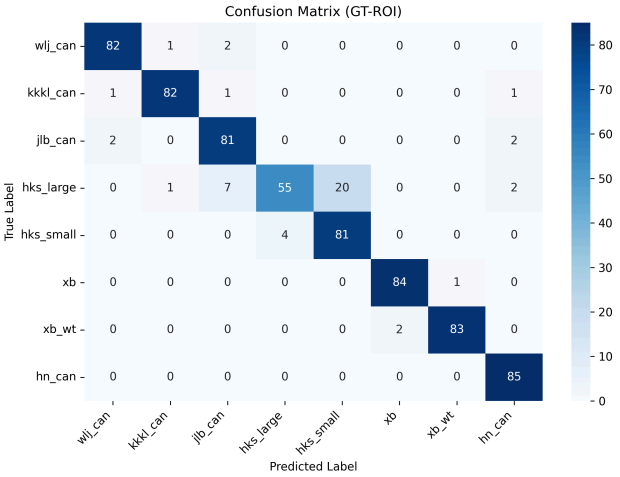


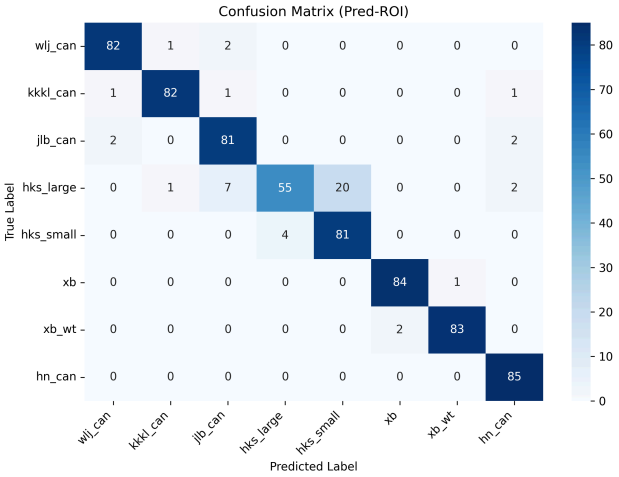
- 训练损失和验证损失随着 epoch 的增加逐渐下降，最终趋于稳定
- 训练准确率和验证准确率随着 epoch 的增加逐渐上升，最终分别达到 100% 和 93.09%

- 早停机制在第 35 个 epoch 触发，保存了验证集表现最佳的模型权重，有效防止了过拟合

5.3 混淆矩阵分析







GT-ROI 和 Pred-ROI 的混淆矩阵对比图显示，绝大多数类别的识别准确率都在 95% 以上，其中雪碧、雪碧无糖和红牛罐装的识别准确率达到了 100%。系统的主要错误集中在“喝开水大瓶”和“喝开水小瓶”的互相混淆上，这是由于两者的包装非常相似，仅在高度上存在差异。

六、错误分析

系统的错误类型统计如下表所示：

错误类型	数量	占比
相似包装混淆	24	51.1%
其他分类错误	23	48.9%
漏检	0	0.0%
误检	0	0.0%
框偏移	0	0.0%

6.1 相似包装混淆

这是系统最主要的错误类型，占所有错误的 51.1%。主要表现为“喝开水大瓶”和“喝开水小瓶”的互相误判。原因是两者的包装设计几乎完全相同，仅在高度上存在细微差异，模型难以从有限的训练数据中学习到区分两者的有效特征。

6.2 其他分类错误

这类错误主要发生在极端光照、角度和遮挡条件下。例如，当商品被手部分遮挡时，模型可能无法提取到足够的特征进行正确分类；当光照过强或过弱时，商品的颜色和纹理特征会发生变化，导致模型误判。

七、结论与展望

7.1 实验结论

本次实验成功实现了“YOLO11 先检测 + ConvNeXt V2 再分类”的两阶段商品识别系统，端到端准确率达到 93.09%，超过了实验要求的 90%。系统在检测阶段表现完美，召回率达到 100%，没有出现漏检和误检的情况。分类阶段的主要瓶颈是相似包装商品的识别，这也是未来系统优化的重点方向。

7.2 改进展望

1. **增加难样本训练**：针对“喝开水大瓶”和“喝开水小瓶”这两个相似类别，收集更多不同角度、不同光照、不同距离下的训练样本，提高模型对细微差异的识别能力。
2. **优化模型结构**：尝试使用更大的模型（如 ConvNeXt V2-Small）或引入注意力机制，增强模型对关键特征的提取能力。
3. **多尺度训练**：采用多尺度训练策略，让模型能够适应不同大小的商品，提高系统的鲁棒性。
4. **两阶段联合优化**：将检测器和分类器进行联合训练，实现端到端的优化，进一步提高系统的整体性能。

八、提交文件清单

1. **代码文件**：
   - `crop.py`： 生成ROI
   - `label.py`： 自动生成 YOLO 格式标注
   - `yolotrain.py`： 训练 YOLO11 检测器
   - `train.py`： 训练 ConvNeXt V2 分类器
   - `result.py`： 端到端系统评估
2. **可视化结果**：
   - `convnextv2_training_curves.png`： ConvNeXt V2 训练曲线
   - `confusion_matrix_comparison.png`： 混淆矩阵对比图
   - `final_evaluation_results/`： 完整的评估结果和错误样本
3. **模型权重**：
   - `convnextv2_drink_classifier_best.pth`： 最佳分类模型权重
   - `runs/yolo11_detection/weights/best.pt`： 最佳检测模型权重